Rust Learning Roadmap

From zero to advanced, with projects

Phase 1: Basics

1-2 weeks

[ ] Install Rust, Cargo, rustc, project layout
[ ] Variables, mutability, types, functions, control flow
[ ] Structs, enums, match, Option

Project: CLI temperature converter and a number guessing game.

Phase 2: Ownership core

3-5 weeks

[ ] Ownership, move vs copy, borrowing, references
[ ] Slices, String vs &str
[ ] Lifetimes (basics)
[ ] Collections: Vec, HashMap, HashSet
[ ] Error handling: Result, ?, custom errors

Project: Word-frequency counter that reads a text file, plus a to-do list CLI that saves to disk.

Phase 3: Abstraction

3-4 weeks

[ ] Modules, crates, visibility
[ ] Generics, traits, trait objects (dyn)
[ ] Iterators and closures
[ ] Operator overloading, Default, From/Into
[ ] Testing: unit, integration, doc tests

Project: Mini grep clone, then a generic stack/queue library with full tests.

Phase 4: Memory and smart pointers

2-3 weeks

[ ] Box, Rc, Arc, Weak
[ ] Cell, RefCell, interior mutability
[ ] Drop, RAII
[ ] Advanced lifetimes

Project: Linked list, binary search tree, and a graph using Rc<RefCell<>>.

Phase 5: Concurrency

3-4 weeks

[ ] Threads, Send and Sync
[ ] Channels, Mutex, RwLock, atomics
[ ] Data parallelism with rayon

Project: Thread pool from scratch, then a parallel file searcher.

Phase 6: Ecosystem and real-world tools

3-4 weeks

[ ] Popular crates: serde, clap, anyhow, thiserror, tracing
[ ] Cargo workspaces, features, publishing a crate
[ ] Benchmarking (criterion), profiling, clippy, rustfmt

Project: JSON-based CLI tool with subcommands, published as a crate.

Phase 7: Async and networking

1-2 months

[ ] async/await, futures, Tokio
[ ] TCP/UDP, HTTP with axum or reqwest
[ ] Pinning, streams, cancellation

Project: Chat server over TCP, then a REST API with a database (sqlx).

Phase 8: Systems and unsafe

1-2 months

[ ] Unsafe Rust, raw pointers
[ ] FFI with C, bindgen
[ ] Memory layout, repr, MaybeUninit
[ ] no_std, embedded basics

Project: Wrap a C library safely, write your own Vec, or blink an LED on a microcontroller.

Phase 9: Advanced

ongoing

[ ] Macros: macro_rules! and procedural macros
[ ] Advanced traits: associated types, GATs, PhantomData
[ ] Type-level programming, zero-cost abstractions
[ ] Lock-free data structures, memory ordering
[ ] Reading compiler and standard library source

Capstone projects (pick one)

[ ] Key-value database with a write-ahead log
[ ] Shell or mini OS kernel
[ ] HTTP server from scratch
[ ] Game with bevy

Resources

• The Rust Book
• Rustlings (exercises)
• Rust by Example
• Rust for Rustaceans (advanced)
• Programming Rust (advanced)